6.31 指令流水线——数据冲突的解决（插入气泡法）

考纲定位：计算机组成原理 · 第 6 章 中央处理器（6.31） | 重要度 ★★★★ | 常见考法：选择题/大题（气泡插入、Stall 信号、Load-Use）

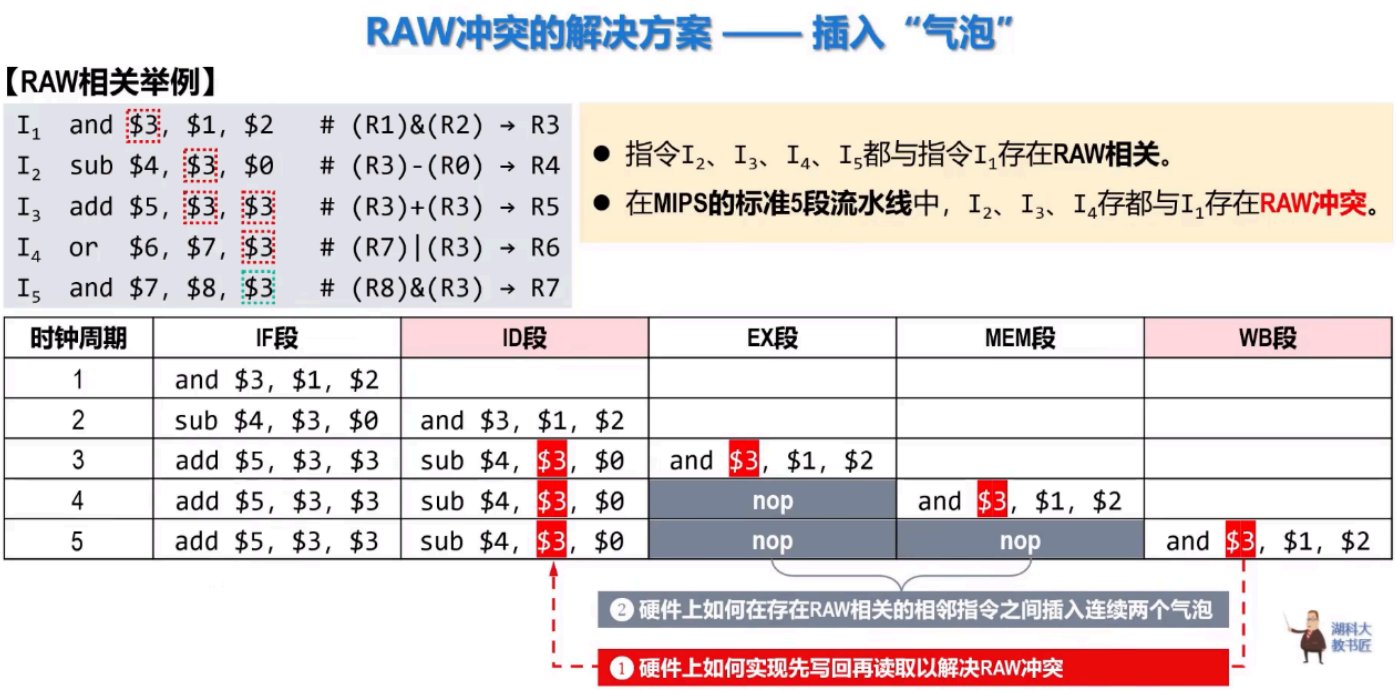
一句话核心：转发来不及就插气泡；Stall 冻结 PC/IF-ID，清空 ID-EX，让消费者晚一拍进 EX

图旁说明：图只保留原图；文字补充 Stall 条件与易错，不重复图中时序

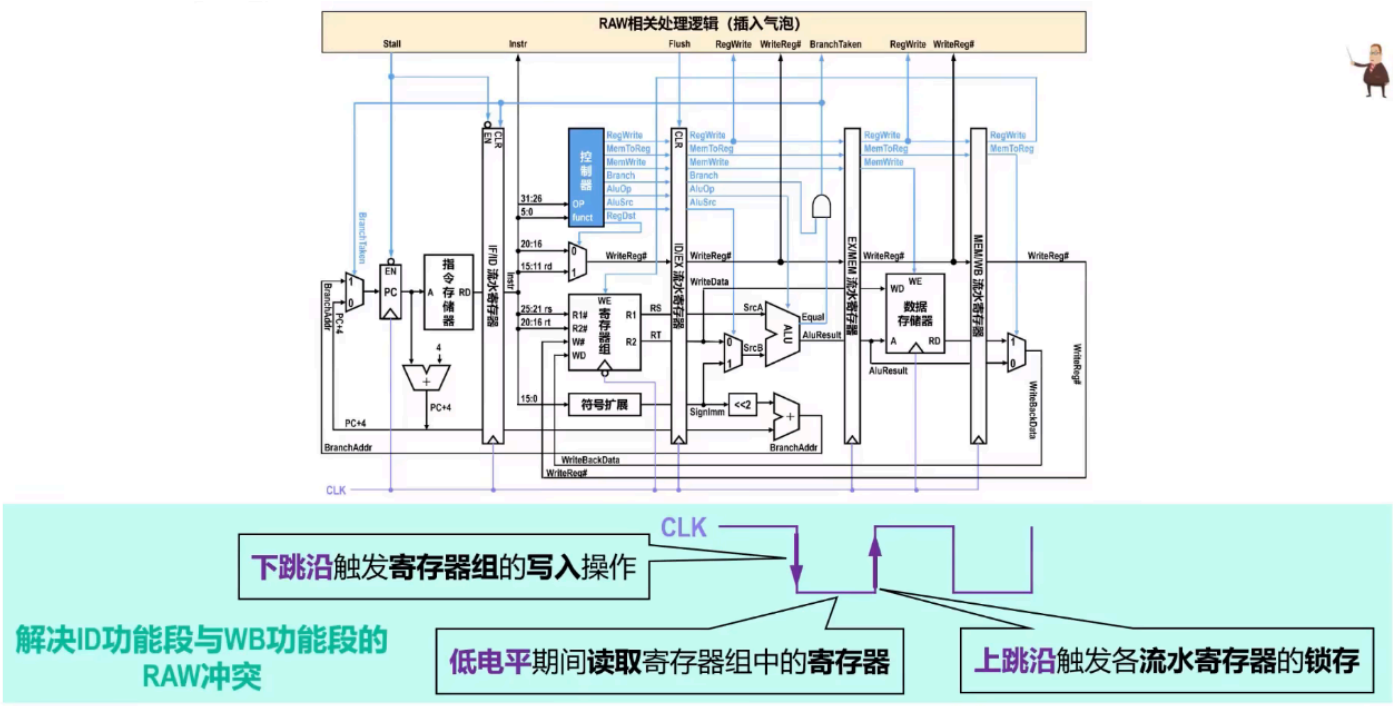
考点/易错

- 气泡不是一条真实指令，是清空 ID/EX 产生的“空操作”
- Stall 条件：ID 段指令需要的源寄存器 = EX/MEM 中 load 的目的寄存器
- Stall 时：PC 和 IF/ID 冻结，ID/EX 清空；不是停整个 CPU
- load-use 只需停 1 拍；普通 ALU 相关靠旁路，不停
- 气泡会降低 CPI；编译器调度可减少气泡

一、举例



二、改造后的指令流水线框架图



三条路径一览

RAW 场景	解决机制	图中对应部分
ALU 指令 → 紧跟的指令	**转发（旁路）**	从 EX/MEM、MEM/WB 连回 ALU 输入的多路选择器
ID 读 ↔ WB 写 同一周期争抢寄存器堆	**半周期分离**	底部紫色标注的时钟时序
Load → 紧跟的指令（Load-Use）	**插入气泡（停顿）**	顶部"RAW相关处理逻辑"模块

路径一：硬件转发（旁路）—— 解决"结果算好了但还没写回"的冲突

场景

I1 的 EX 段已经算出了结果，但还没到 WB 段写回寄存器堆；此时 I2 已经进入 EX 段需要这个结果

图中怎么体现

看 ALU 左侧的两个多路选择器 **SrcA** 和 **SrcB**：

- 正常情况下，ALU 的操作数来自**寄存器堆**（即 ID 段读出的 Rs、Rt）；
- 但图中有**额外的线**从 **EX/MEM 流水寄存器**和 **MEM/WB 流水寄存器**的输出端连回到 SrcA 和 SrcB 的输入端

这两条线就是**旁路（forwarding path）**：

```
EX/MEM 中的 AluResult  ──→ SrcA/SrcB 选择器 ──→ ALU输入
MEM/WB 中的 AluResult  ──→ SrcA/SrcB 选择器 ──→ ALU输入
MEM/WB 中的 MemoryData ──→ SrcA/SrcB 选择器 ──→ ALU输入
```

工作原理：

- 转发检测逻辑（图中未展开，但由 RAW 相关处理逻辑的一部分承担）判断"后一条指令需要的源寄存器编号 = 前一条指令的目的寄存器编号"；
- 若命中，就把 SrcA/SrcB 的选择控制信号设为"取旁路数据"，**绕过寄存器堆**，直接把中间结果喂给 ALU

一句话

"与其等 WB 写回再读，不如把中间结果直接抄近道送给后面的人

路径二：半周期分离 —— 解决 ID 读与 WB 写的"撞车"

场景

I1 在 WB 段要把结果写入寄存器堆，同时 I2 在 ID 段要从同一个寄存器堆读出数据。如果都在同一个时钟周期完成，就会出现"先读还是先写"的冲突

图中怎么体现（看底部紫色标注）

这是理解整张图最关键也最容易忽视的部分。它把**一个时钟周期切成两半**来用：

```
时钟信号：  ──高──┐     ┌──高──┐     ┌──
                  └─────┘      └─────┘
                  ↑ 低电平 ↓     ↑ 低电平 ↓

一个周期内：
┌── 上升沿触发流水线寄存器锁存（IF/ID, ID/EX, EX/MEM, MEM/WB）
├── 低电平期间读取寄存器堆
└── 下降沿触发寄存器堆写入（WB段写入）
```

关键点：WB 写发生在下降沿，ID 读发生在低电平期间。

但等等——这看起来 WB 写（下降沿）比 ID 读（低电平）还晚啊？那怎么解决冲突？

真正的奥秘在于"跨周期"：

```
周期 N:                周期 N+1:
I1(WB段) ─下降沿写寄存器堆 ──→  完成写入
                                  I2(ID段) ─低电平期间读寄存器堆
```

I1 在周期 N 的**下降沿**就已经把数据写进去了，I2 在周期 N+1 的**低电平期间**才读。所以读的时候数据已经在了，**不存在冲突**

但如果 I1 和 I2 在同一个周期内分别处于 WB 和 ID（即 I2 紧跟在 I1 后面第 4 拍），那么：

```
周期 5:
```

```
WB(I1) 下降沿写 —— 此时 ID(I2) 还在低电平读
```

这里图中标注说"解决ID功能段与WB功能段的RAW冲突"，意思就是：**由于写入发生在下降沿、读取在低电平期间，硬件上保证了先写后读的顺序**

实际上更准确的说法是：**寄存器堆被设计为"前半周期写、后半周期读"或"下降沿写、低电平读"**，在同一个时钟周期内，写操作总是先于读操作完成，因此不会出现读到旧值的问题

一句话

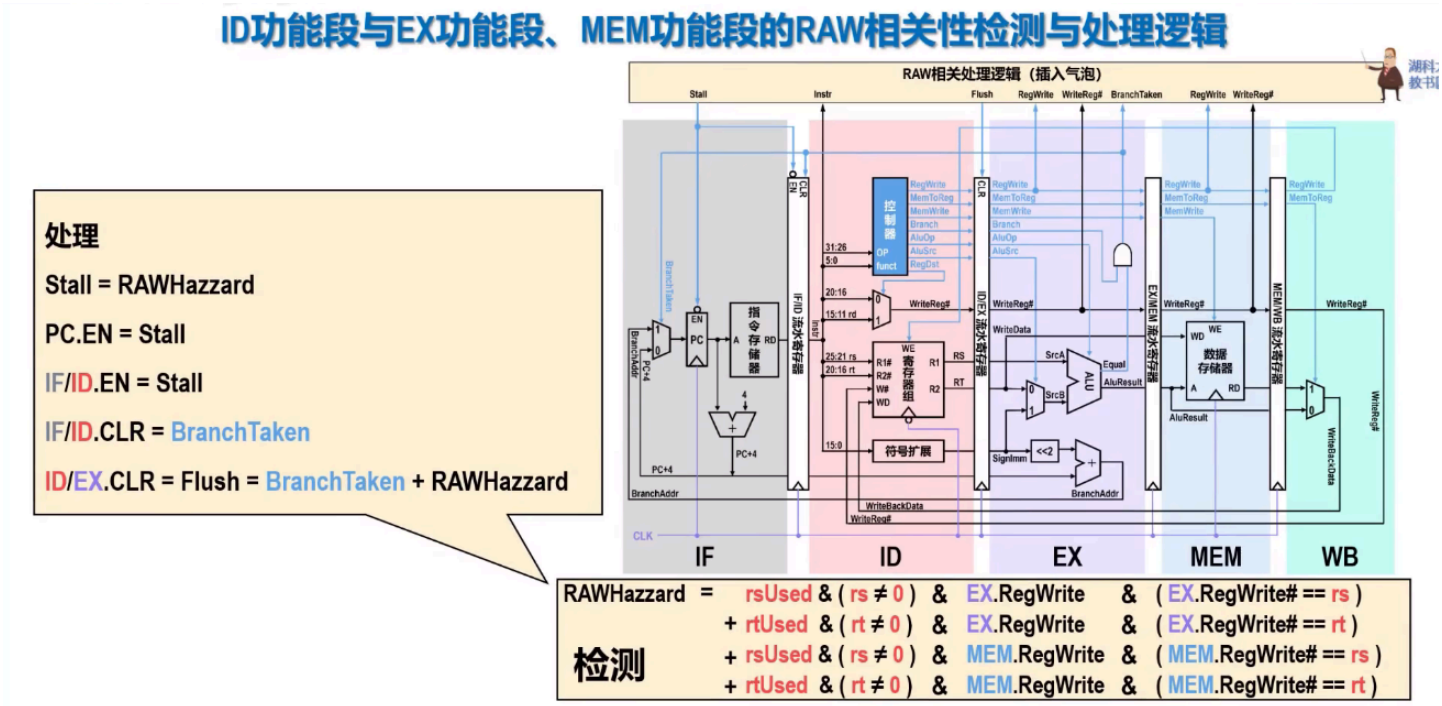
"把写放在下降沿、读放在低电平，让写先于读完成，就不需要停顿"

路径三：插入气泡（Stall）—— 解决 Load-Use 冒险

场景

I1 是 `LW R1, 0(R2)`，数据要到 **MEM 段末**才从存储器取出来；I2 紧跟在 I1 后面，在 EX 段需要 R1 的值。即使有转发，数据也来不及（晚一拍）

图中怎么体现



==RAW相关处理逻辑（插入气泡）==就是做这件事的控制器：

它监测以下信号：

- 当前 ID 段的指令是否需要某个寄存器（Rs/Rt）
- 上一拍的指令（在 EX/MEM 中）是不是 `LW` 且目的寄存器（rt）正好是当前指令需要的源寄存器

如果两个条件同时满足 → **产生 Stall 信号**

Stall 信号的作用：

1. **暂停 PC 和 IF/ID 寄存器**（冻结取指和译码，不取新指令进来）

2. **清空 ID/EX 寄存器**（插入一个气泡/no-op 指令到 EX 段）

效果就是 I2 的 EX 段被推迟一拍：

```
正常流水（无停顿）：
I1(LW):    IF   ID   EX   MEM  WB
I2:             IF   ID   EX   MEM  WB
                          ↑ I2的EX在第4拍，但load数据第4拍末才出来

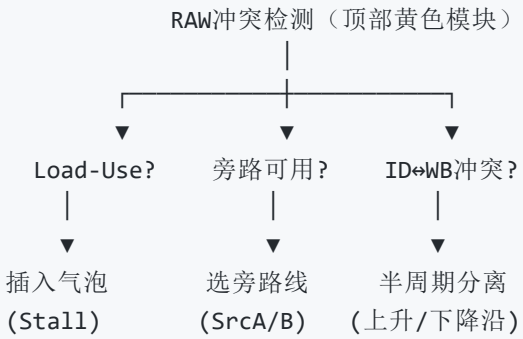
插入气泡后：
I1(LW):    IF   ID   EX   MEM  WB
I2:             IF  --stall--  ID   EX   MEM  WB
                          ↑ 插入一个气泡（空操作）
                          ↑ I2的EX推迟到第5拍，刚好能用转发数据
```

图中可以看到 Stall 信号线连接到 PC 的 EN（使能）端和 IF/ID 流水寄存器，控制它们暂停更新

一句话

"转发也来不及的时候，就只能让后面的指令等一拍"

三者的协作关系



三种机制不是互斥的，而是**同时工作**的：

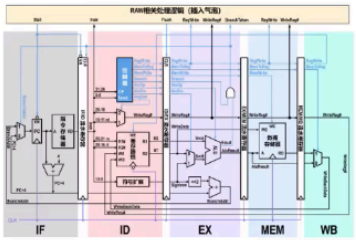
- 旁路解决大部分 ALU 的 RAW → 不需要停顿
- 半周期分离解决寄存器堆读写冲突 → 不需要额外逻辑
- 插入气泡兜底 Load-Use → 唯一必须停顿的场景

三、插入气泡法解题举例

【练习】请给出该程序在右图所示气泡流水线中运行的时空图。

```
I1  lw $2, 8($1)          # M[R(1)+8] → R2
I2  add $3, $2, $4        # (R2)+(R4) → R3
I3  sub $5, $6, $7        # (R6)-(R7) → R5
I4  or  $8, $3, $9        # (R3)|(R9) → R8
I5  and $10, $8, $11      # (R8)&(R11) → R10
```





时钟周期	IF段	ID段	EX段	MEM段	WB段
1	lw $2, 8($1)				
2	add $3, $2, $4	lw $2, 8($1)			
3	sub $5, $6, $7	add $3, $2, $4	lw $2, 8($1)		
4	sub $5, $6, $7	add $3, $2, $4	nop	lw $2, 8($1)	
5	sub $5, $6, $7	add $3, $2, $4	nop	nop	lw $2, 8($1)
6	or $8, $3, $9	sub $5, $6, $7	add $3, $2, $4	nop	nop
7	and $10, $8, $11	or $8, $3, $9	sub $5, $6, $7	add $3, $2, $4	nop
8	and $10, $8, $11	or $8, $3, $9	nop	sub $5, $6, $7	add $3, $2, $4
9	下一条指令	and $10, $8, $11	or $8, $3, $9	nop	sub $5, $6, $7
10	下一条指令	and $10, $8, $11	nop	or $8, $3, $9	nop
11	下一条指令	and $10, $8, $11	nop	nop	or $8, $3, $9